Solution: Merge Strings Alternately

Problem Statement

Given two strings, `word1` and `word2`, return the `merged string` after merging each character of both strings in alternate order, starting with the `word1`. If one string is longer than the other, append the remaining characters of the longer string to the end of the merged string.

Examples

- **Example 1**:
  - **Input**: word1: `"dog"`, word2: `"cat"`
  - **Expected Output**: `"dcoatg"`
  - **Justification**: The characters from both strings are alternated, starting with the first character of the first string. Since both strings are of equal length, no extra characters remain at the end.
- **Example 2**:
  - **Input**: word1: `"hello"`, word2: `"hi"`
  - **Expected Output**: `"hheillo"`
  - **Justification**: After alternating characters from both strings, the remaining characters from the longer string ("hello") are appended to the end.
- **Example 3**:
  - **Input**: word1: `"abc"`, word2: `"xyz123"`
  - **Expected Output**: `"axbycz123"`
  - **Justification**: Characters are alternated, and since the second string is longer, the remaining characters ("123") are appended to the end.

Solution

To solve this problem, we'll iterate over both strings simultaneously, appending one character from each string to the result until we reach the end of one of the strings. This approach ensures that characters are merged in an alternating fashion, preserving the order from both input strings. After completing the alternating merge, if one string is longer, we append the remaining characters of the longer string to the merged result. This method is effective because it directly addresses the problem's requirements with a straightforward iteration, making it easy to understand and implement. By iterating over the strings simultaneously, we minimize the time complexity, ensuring that each character is visited only once, which makes the algorithm efficient for strings of varying lengths.

Step-by-Step Algorithm

- Initialize an empty string `result` to store the merged output.
- Determine the length of the shorter string to ensure the alternating merge doesn't exceed the shorter string's length.
- Use a loop to iterate through both strings up to the length of the shorter string. During each iteration:
  - Append the current character from the first string to `result`.
  - Append the current character from the second string to `result`.
- After completing the alternating merge, check if any string has remaining characters. Append the remaining characters from the longer string to `result`.
- Return the `result` string.

Algorithm Walkthrough

Consider the input: word1: "abc", word2: "xyz123"

1. **Initialize Variables**:
   - `result` is an empty string where the merged output will be stored.
   - Determine the length of the shorter string. In this case, `word1` has a length of 3, and `word2` has a length of 6. So, the length of the shorter string is 3.
2. **Start Looping Through Both Strings**:
   - For `i = 0` (first iteration):
     - Append the first character of `word1` ("a") to `result`. Now, `result` = "a".
     - Append the first character of `word2` ("x") to `result`. Now, `result` = "ax".
   - For `i = 1` (second iteration):
     - Append the second character of `word1` ("b") to `result`. Now, `result` = "axb".
     - Append the second character of `word2` ("y") to `result`. Now, `result` = "axby".
   - For `i = 2` (third iteration):
     - Append the third character of `word1` ("c") to `result`. Now, `result` = "axbyc".
     - Append the third character of `word2` ("z") to `result`. Now, `result` = "axbycz".
3. **Check for Remaining Characters**:
   - At this point, `word1` is fully iterated through, but `word2` still has characters left ("123").
   - Since `word1` is shorter and fully merged, we append the remaining characters of `word2` to `result`. Now, `result` = "axbycz123".
4. **Return the Result**:
   - The final merged string is "axbycz123", which is stored in `result`.
   - Return `result`.

Code

Java

```java
public class Solution {
    // Method to merge two strings alternately
    public String mergeAlternately(String word1, String word2) {
        StringBuilder result = new StringBuilder(); // StringBuilder to store the result
        int minLength = Math.min(word1.length(), word2.length()); // Determine the short
        // Loop through both strings up to the length of the shorter string
        for (int i = 0; i < minLength; i++) {
            result.append(word1.charAt(i)); // Append character from word1
            result.append(word2.charAt(i)); // Append character from word2
        }
        // Append remaining characters from the longer string, if any
        if (word1.length() > minLength) {
            result.append(word1.substring(minLength));
        } else if (word2.length() > minLength) {
            result.append(word2.substring(minLength));
        }
        return result.toString(); // Convert StringBuilder to String and return
    }

    // Main method to test the algorithm with example inputs
    public static void main(String[] args) {
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity**: $O(N + M)$, where N is the length of the first string and M is the length of the second string. This is because the algorithm iterates through both strings once.
- **Space Complexity**: $O(N + M)$, due to the storage requirements for the resulting string which combines characters from both input strings.

← Previous
Merge Strings Alternately (Easy)

Next →
Min Cost Climbing Stairs (Easy)

☐ Mark as Completed